

Oriol Chiva Hidalgo

Trabajo Obligatorio. *Machine learning*

Septiembre 2026

GRUPO SAN VALERO
SEAS, Estudios Superiores Abiertos
ZARAGOZA

TRABAJO OBLIGATORIO
Propuesta

Propuesta de trabajo

Vamos a plantear la realización del trabajo obligatorio con un conjunto de datos muy conocido: Las flores Iris que, como sabemos, pueden ser de tres categorías distintas. Sin embargo, para aquellos alumnos que deseen trabajar otro tipo de datos, lo pueden solicitar.

El trabajo constará de las siguientes fases:

1. Hacer una tabla con las clases, métodos y parámetros vistos en el *paquete Scikit learn* en algoritmos supervisados y no supervisados.
2. Descripción y carga del conjunto de datos.
3. Análisis estadístico y representación gráfica de los datos.
4. Elección de un modelo de clasificación supervisado y descripción del mismo.
5. Entrenamiento, evaluación y validación del modelo supervisado.
6. Elección de un modelo de clasificación no supervisado.
7. Entrenamiento, evaluación y validación del modelo no supervisado.
8. Conclusiones.

Objetivos del trabajo

Con el presente trabajo se pretende:

1. Repasar los conceptos vistos en las distintas unidades del curso.
2. Realizar un trabajo de clasificación que requiera hacer uso de lo aprendido en la mayor parte de las Unidades del curso.
3. Enfrentarse a un proyecto completo de *Machine Learning.*

Bibliografía

- Manual de la asignatura.
- Páginas web citadas en la asignatura.

Criterios de evaluación

La evaluación es una componente fundamental de la formación. Este trabajo obligatorio formará parte de tu calificación final. En esta tabla, se resumen los aspectos a valorar y el porcentaje que representa cada uno de los mismos.

CONTENIDOS GENERALES	**10 %**
Descripción básica del trabajo y conclusiones	10
TEMAS DE ESPECIALIDAD	**80 %**
Tabla resumen de los módulos, funciones y parámetros	10
Descripción y carga del conjunto de datos	10
Análisis estadístico y representación gráfica de los datos	10
Elección y descripción de un modelo supervisado	10
Entrenamiento, evaluación y validación del modelo supervisado	15
Elección y descripción de un modelo no supervisado	10
Entrenamiento, evaluación y validación del modelo no supervisado	15
OTRAS APORTACIONES	**10 %**
Comentar de forma clara el código	5
Mostrar los resultados en todas las opciones posibles	5

Fecha límite de recepción de trabajos

La fecha de entrega deberá ser anterior a la fecha fin de matrícula.

Ficha de Corrección del Trabajo

(Espacio reservado para anotaciones del profesor).

Profesor:	
Alumno (Código / Nombre):	
Fecha de Entrega:	Fecha de Calificación:

Observaciones sobre el trabajo

Este espacio está reservado para que el profesor titular describa anotaciones que considera importantes sobre la realización del trabajo

Formato de presentación

1. Se presentará en formato informático.
2. Se dejará libre elección al alumno para el diseño y estilo del documento que quiera crear.
3. En caso de que el trabajo requiera archivos externos, estos deberán entregarse junto al trabajo.
4. Si el trabajo consta de varios archivos, deberá enviarse en un solo fichero comprimido.
5. Si el sistema no te permite adjuntar el trabajo obligatorio en el campo indicado porque excede del tamaño permitido, deberás ponerte en contacto con tu tutor del estudio con el fin de acordar el procedimiento que seguir para el envío del mismo. Además, deberás adjuntar desde la plataforma en el campo "sube tu ejercicio" un índice del trabajo realizado a modo de resumen para una adecuada corrección y funcionamiento del sistema informático. En ese documento debe constar:
   - El medio escogido para el envío.
   - La fecha de envío.
   - De qué consta el envío (por ejemplo; un documento word, tres fotos,...).
   - El *link* para la descarga por parte del profesor.

Desarrollo del trabajo

Introducción y objetivo

El objetivo de este trabajo es aplicar de forma completa las etapas básicas de un problema de Machine Learning utilizando el conjunto de datos Iris. Se seguirá la propuesta del Trabajo Obligatorio de SEAS: resumir las principales clases y métodos de Scikit-learn estudiados, cargar y analizar los datos, seleccionar un modelo supervisado y otro no supervisado, entrenarlos, evaluarlos y extraer conclusiones.

El conjunto Iris es especialmente adecuado porque contiene tres especies de flores y cuatro características numéricas: longitud y anchura del sépalo, y longitud y anchura del pétalo. Esto permite trabajar tanto clasificación supervisada como agrupamiento no supervisado y representar gráficamente las relaciones entre las variables.

Tabla resumen de clases, métodos y parámetros de Scikit-learn

En Scikit-learn los modelos se utilizan mediante estimadores. El esquema general estudiado en el temario consiste en elegir una clase, crear una instancia con los hiperparámetros deseados, organizar los datos en una matriz de características X y un vector objetivo y, entrenar mediante fit() y aplicar posteriormente predict(), transform() o los métodos de evaluación correspondientes.

Clase	Módulo	Uso	Parámetros destacados	Métodos / atributos
LinearRegression	sklearn.linear_model	Regresión supervisada	fit_intercept	fit(), predict(), score()
LogisticRegression	sklearn.linear_model	Clasificación supervisada	C, solver, max_iter	fit(), predict(), predict_proba(), score()
SVC	sklearn.svm	Clasificación supervisada	C, kernel, gamma	fit(), predict(), score()
SVR	sklearn.svm	Regresión supervisada	C, kernel, epsilon	fit(), predict(), score()
MLPClassifier	sklearn.neural_network	Clasificación supervisada	hidden_layer_sizes, solver, alpha, random_state	fit(), predict(), predict_proba(), score()
KNeighborsClassifier	sklearn.neighbors	Clasificación por vecinos	n_neighbors, weights, algorithm	fit(), predict(), score()
RadiusNeighborsClassifier	sklearn.neighbors	Clasificación por vecinos	radius, weights, algorithm	fit(), predict(), score()
NearestCentroid	sklearn.neighbors	Clasificación por centroide	metric	fit(), predict()
KMeans	sklearn.cluster	Clustering no supervisado	n_clusters, random_state	fit(), predict(); labels_, inertia_, n_iter_, cluster_centers_
DBSCAN	sklearn.cluster	Clustering no supervisado	eps, min_samples	fit(), fit_predict(); labels_
PCA	sklearn.decomposition	Reducción de dimensionalidad	n_components	fit(), transform(), fit_transform()

Descripción y carga del conjunto de datos

Iris contiene 150 muestras repartidas en tres especies: setosa, versicolor y virginica. Cada muestra se describe mediante cuatro variables cuantitativas continuas medidas en centímetros. Al estar etiquetadas las especies, el conjunto puede utilizarse directamente para clasificación supervisada; si se ignoran esas etiquetas, también permite estudiar clustering.

Scikit-learn incorpora Iris entre sus conjuntos de datos estándar. El objeto devuelto por load_iris() contiene data con las características, target con las etiquetas y target_names con los nombres de las especies.

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
X = iris.data
y = iris.target

df = pd.DataFrame(X, columns=iris.feature_names)
df["species"] = [iris.target_names[i] for i in y]

print(df.head())
print(df.shape)
print(df["species"].value_counts())
```

La matriz X tiene 150 filas y 4 columnas. El vector y contiene 150 etiquetas codificadas como 0, 1 y 2. El conjunto está equilibrado: hay 50 muestras de cada especie.

Análisis estadístico y representación gráfica

Antes de construir los modelos se realiza un análisis exploratorio. Según el temario, la visualización es importante porque permite detectar tendencias, patrones y relaciones entre variables. En Iris se observa especialmente que las medidas del pétalo presentan una separación clara entre setosa y las otras dos especies.

Variable	Media	Desv. típica	Mínimo	Máximo
Longitud sépalo	5,843	0,828	4,3	7,9
Anchura sépalo	3,057	0,436	2,0	4,4
Longitud pétalo	3,758	1,765	1,0	6,9
Anchura pétalo	1,199	0,762	0,1	2,5

```
print(df.describe())
```

Código utilizado para obtener la Figura 1. Seaborn representa cada flor según la longitud y la anchura de su pétalo. El parámetro hue="species" asigna una categoría visual distinta a cada especie, lo que permite observar la separación entre las clases.

```
sns.scatterplot(data=df, x="petal length (cm)", y="petal width (cm)", hue="species") plt.xlabel("Longitud del pétalo (cm)") plt.ylabel("Anchura del pétalo (cm)") plt.title("Relación entre longitud y anchura del pétalo") plt.show()
```

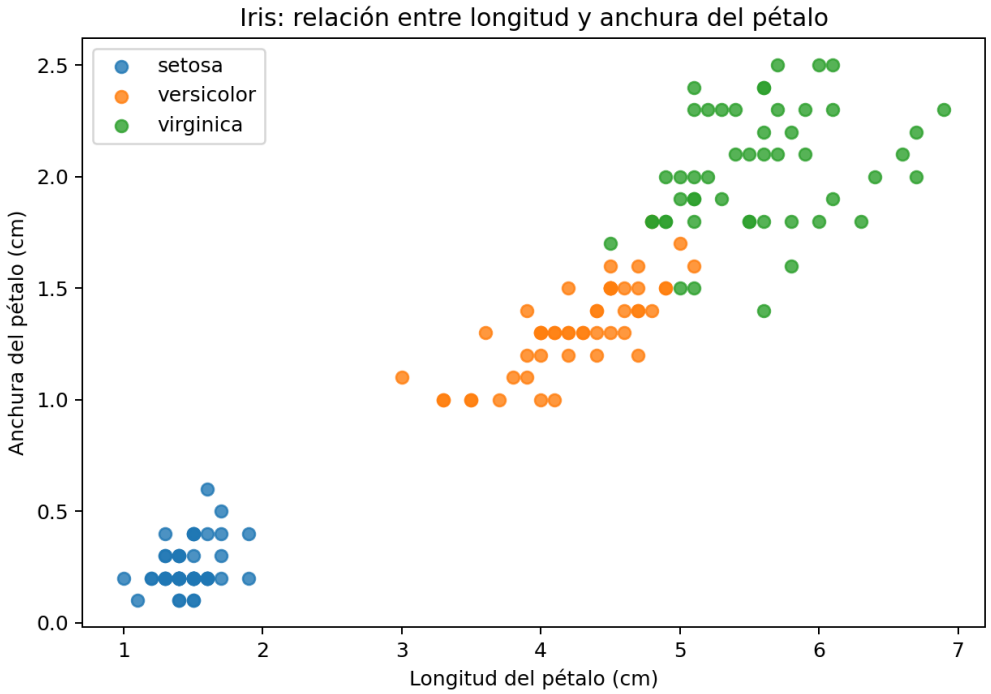


Figura 1. Relación entre longitud y anchura del pétalo.

Código utilizado para obtener la Figura 2. El diagrama de caja compara la distribución de la longitud del pétalo para las tres especies. Permite visualizar la mediana, la dispersión de los datos y las diferencias existentes entre las clases.

```
sns.boxplot(data=df, x="species", y="petal length (cm)") plt.xlabel("Especie")
plt.ylabel("Longitud del pétalo (cm)") plt.title("Longitud del pétalo por especie")
plt.show()
```

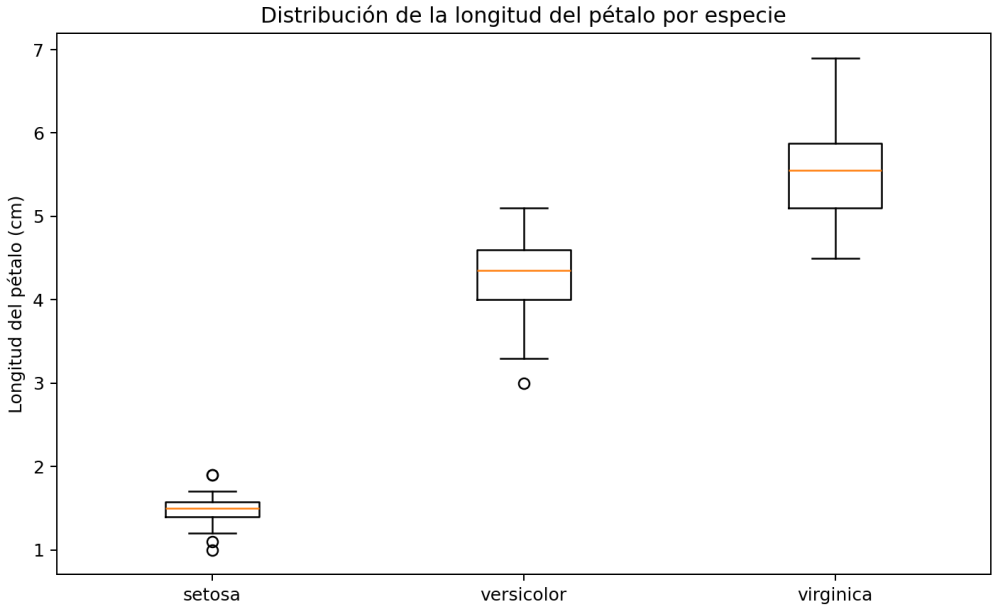


Figura 2. Diagrama de caja de la longitud del pétalo por especie.

Setosa queda claramente separada mediante las variables del pétalo. Versicolor y virginica presentan cierto solapamiento, aunque mantienen diferencias apreciables. Por ello, las cuatro características se conservarán para el entrenamiento.

Elección y descripción del modelo supervisado

Se ha elegido una máquina de vectores soporte para clasificación mediante la clase SVC. El temario presenta las máquinas de vectores soporte dentro de los algoritmos supervisados. En clasificación, el modelo aprende a separar las clases a partir de ejemplos previamente etiquetados.

Se utilizará un kernel lineal. El parámetro C controla la penalización de los errores de clasificación. Antes del SVC se estandarizan las características con StandardScaler, de forma que las distintas escalas no condicionen las distancias y el ajuste del modelo.

La elección es adecuada para Iris porque se trata de un problema multiclase con pocas muestras, cuatro variables numéricas y clases que presentan una separación importante en el espacio de características.

Entrenamiento, evaluación y validación del modelo supervisado

Los datos se dividen en entrenamiento y ensayo. Se reserva un 30 % para ensayo y se utiliza random_state=42 para poder reproducir la división. La opción stratify=y conserva la proporción de las tres especies.

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

modelo = make_pipeline(
    StandardScaler(),
    SVC(kernel="linear", C=1)
)

modelo.fit(X_train, y_train)
predicciones = modelo.predict(X_test)

print(confusion_matrix(y_test, predicciones))
print(classification_report(y_test, predicciones))

scores = cross_val_score(modelo, X, y, cv=5)
print(scores)
print(scores.mean())
```

Resultado sobre el conjunto de ensayo: 41 muestras correctamente clasificadas de 45, lo que corresponde a una exactitud del 91,11 %. Setosa se clasifica correctamente en los 15 casos. Los errores aparecen únicamente entre versicolor y virginica, coherentemente con el solapamiento observado en las gráficas.

Real \ Pred.	Setosa	Versicolor	Virginica
Setosa	15	0	0
Versicolor	0	14	1
Virginica	0	3	12

Código utilizado para obtener la Figura 3. A partir de las predicciones del modelo SVC se construye la matriz de confusión. Las filas representan las clases reales y las columnas las clases predichas, de modo que se pueden localizar visualmente los aciertos y los errores de clasificación.

```
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_predictions(     y_test,     predicciones,
display_labels=iris.target_names ) plt.title("Matriz de confusión - SVC") plt.show()
```

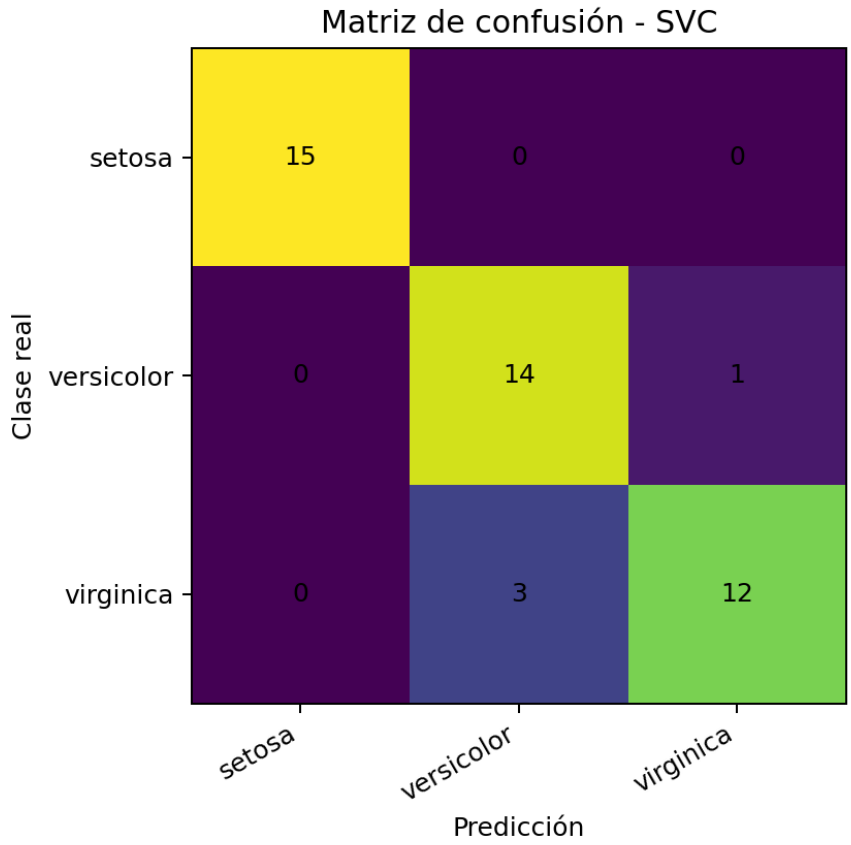


Figura 3. Matriz de confusión obtenida con el modelo SVC.

Para complementar una única partición entrenamiento/ensayo se realiza validación cruzada de cinco particiones. Las exactitudes obtenidas son 96,67 %, 100 %, 93,33 %, 93,33 % y 100 %, con una media del 96,67 %. La desviación entre particiones es pequeña (aproximadamente 2,98 puntos porcentuales), lo que indica un comportamiento estable en este conjunto de datos.

La matriz de confusión y el informe de clasificación son precisamente dos de las herramientas de evaluación tratadas en el temario para analizar la corrección de un clasificador.

Elección y descripción del modelo no supervisado

Como modelo no supervisado se selecciona K-Means. Los algoritmos de clustering trabajan con datos no etiquetados e intentan descubrir grupos de muestras similares. K-Means representa cada agrupamiento mediante un centroide y asigna las muestras al grupo correspondiente.

Se fija n_clusters=3 porque sabemos que Iris contiene tres especies y porque interesa comprobar hasta qué punto la estructura descubierta sin usar las etiquetas coincide con esas tres categorías. Se utiliza random_state=0 para reproducibilidad. El temario destaca labels_, inertia_, n_iter_ y cluster_centers_ como atributos útiles de KMeans.

También se aplica el método del codo: se entrena K-Means con distintos valores de K y se representa la inercia. El objetivo es observar cómo disminuye la suma de las distancias cuadráticas de los puntos a sus centroides al aumentar el número de grupos.

Entrenamiento, evaluación y validación del modelo no supervisado

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

escalador = StandardScaler()
X_escalado = escalador.fit_transform(X)

modelo_kmeans = KMeans(
    n_clusters=3,
    random_state=0,
    n_init=10
)

clusters = modelo_kmeans.fit_predict(X_escalado)

print(modelo_kmeans.labels_)
print(modelo_kmeans.cluster_centers_)
print(modelo_kmeans.inertia_)
print(modelo_kmeans.n_iter_)

print("Silhouette:", silhouette_score(X_escalado, clusters))
```

Código utilizado para obtener la Figura 4. Se entrena K-Means con distintos valores de K y se guarda la inercia de cada modelo. Al representar esos valores se obtiene el método del codo, que ayuda a estudiar qué número de agrupamientos resulta razonable.

```
inercias = []  for k in range(1, 9):     modelo_k = KMeans(n_clusters=k,
random_state=0, n_init=10)     modelo_k.fit(X_escalado)
inercias.append(modelo_k.inertia_)  plt.plot(range(1, 9), inercias, marker="o")
plt.xlabel("Número de agrupamientos K") plt.ylabel("Inercia") plt.title("Método del
codo para K-Means") plt.show()
```

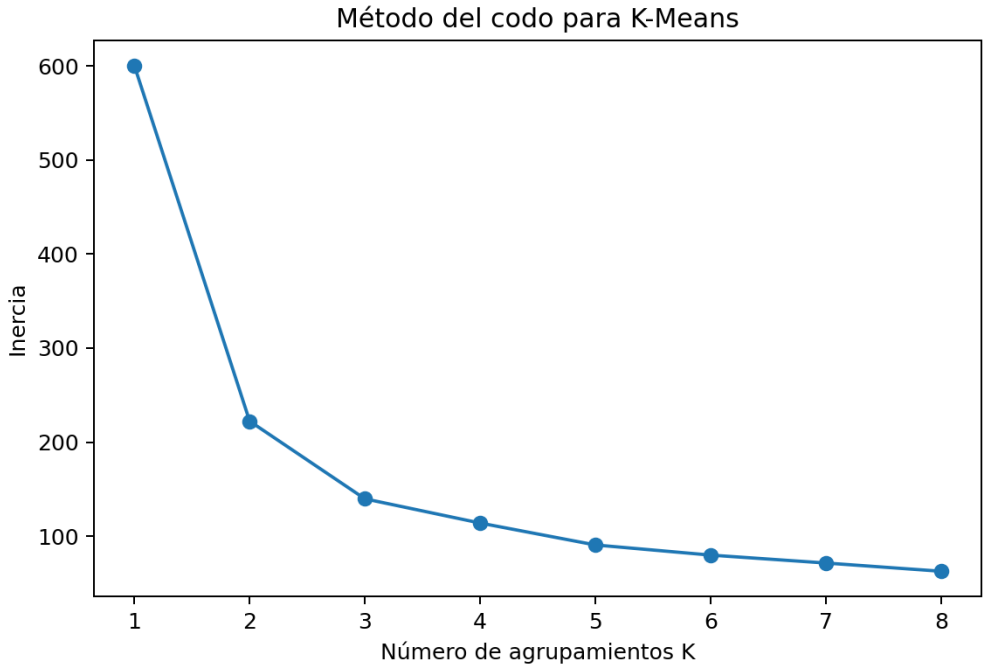


Figura 4. Método del codo a partir de la inercia de K-Means.

Con K=3, el coeficiente silhouette obtenido es aproximadamente 0,460. Este valor indica que existe estructura de agrupamiento, aunque los grupos no están completamente separados. Esto coincide con lo observado gráficamente: setosa forma un grupo muy definido, mientras que versicolor y virginica están más próximas.

Para representar las cuatro dimensiones en un plano se utiliza PCA con dos componentes principales. El temario utiliza precisamente PCA para reducir Iris y poder visualizar su estructura en dos dimensiones.

Código utilizado para obtener la Figura 5. Como Iris tiene cuatro características, PCA reduce los datos estandarizados a dos componentes principales para poder representarlos en un plano. Después, cada punto se muestra según el cluster que K-Means le ha asignado.

```
pca = PCA(n_components=2) X_pca = pca.fit_transform(X_escalado)  plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters) plt.xlabel("Componente principal 1")
plt.ylabel("Componente principal 2") plt.title("K-Means sobre Iris") plt.show()
```

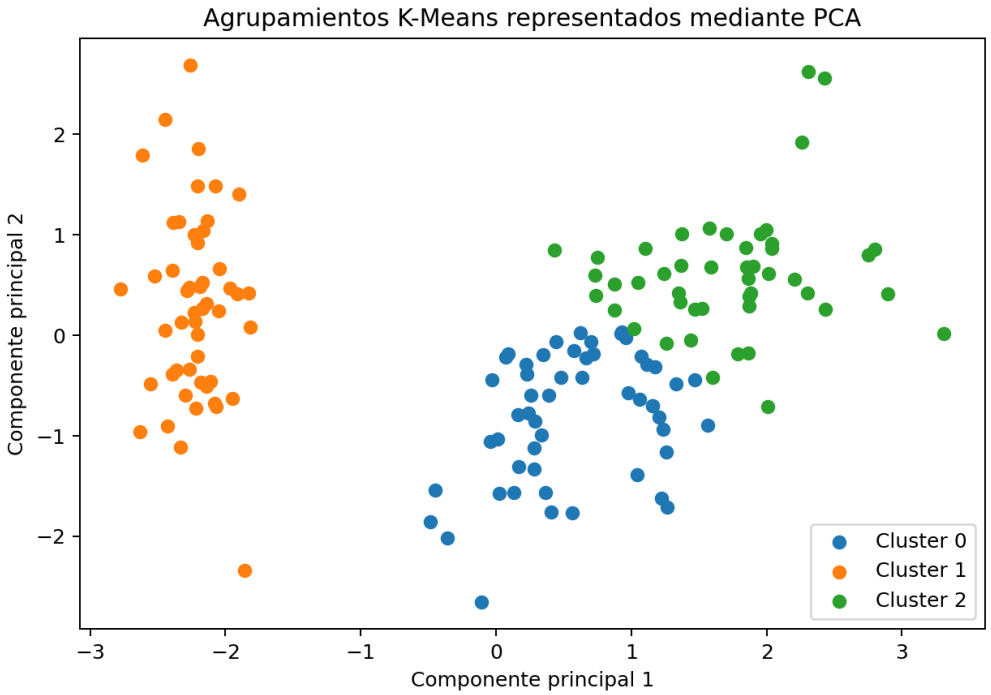


Figura 5. Resultado de K-Means proyectado en dos componentes principales.

Como comprobación externa, y únicamente después de realizar el clustering, se pueden comparar los grupos obtenidos con las etiquetas reales. Los números asignados por K-Means a los clusters son arbitrarios, por lo que primero se hace corresponder cada cluster con la especie mayoritaria. Con esa correspondencia, 125 de las 150 muestras coinciden con su especie real, es decir, un 83,33 %. Setosa queda completamente separada; la mayor confusión vuelve a producirse entre versicolor y virginica.

Especie real \ Grupo asociado	Setosa	Versicolor	Virginica
Setosa	50	0	0
Versicolor	0	39	11
Virginica	0	14	36

Esta comparación no convierte K-Means en un algoritmo supervisado: las etiquetas reales no se utilizan durante el entrenamiento, sino solamente al final para interpretar el resultado obtenido.

Conclusiones

El conjunto Iris permite comprobar de forma sencilla las diferencias entre aprendizaje supervisado y no supervisado. En el modelo supervisado, SVC utiliza las especies conocidas durante el entrenamiento y obtiene un rendimiento elevado. En la partición de ensayo utilizada alcanza un 91,11 % de exactitud y la validación cruzada de cinco particiones obtiene una media del 96,67 %.

En el aprendizaje no supervisado, K-Means no recibe las especies y forma los grupos exclusivamente a partir de las cuatro medidas. El resultado reproduce bien la estructura general del conjunto, especialmente para setosa, aunque la separación entre versicolor y virginica es menos clara. El coeficiente silhouette de 0,460 y la comparación externa del 83,33 % muestran esa diferencia.

Las representaciones gráficas permiten entender los resultados antes y después del modelado. Las variables relacionadas con el pétalo son especialmente útiles para distinguir las especies, mientras que el solapamiento de versicolor y virginica explica buena parte de los errores observados en ambos enfoques.

En conjunto, el trabajo reproduce el flujo estudiado en el temario: carga y exploración de los datos, elección de estimadores e hiperparámetros, organización de X e y, entrenamiento con fit(), predicción, evaluación y representación de los resultados. La principal conclusión es que disponer de etiquetas aporta una ventaja clara cuando el objetivo es predecir una clase conocida, mientras que el clustering resulta útil para descubrir estructura cuando dichas etiquetas no están disponibles.

Código completo

El siguiente script reúne todas las operaciones realizadas en el trabajo. Incluye también el código exacto necesario para generar las Figuras 1, 2, 3, 4 y 5, de forma que los resultados gráficos puedan reproducirse ejecutando el programa completo.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay,
    silhouette_score
)
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA


# 1. CARGA DEL CONJUNTO DE DATOS IRIS

iris = load_iris()

X = iris.data
y = iris.target

df = pd.DataFrame(X, columns=iris.feature_names)
df["species"] = [iris.target_names[i] for i in y]

# Primeras filas del conjunto de datos
print(df.head())

# Información estadística
print(df.describe())
# 2. FIGURA 1 - GRÁFICO DE DISPERSIÓN

sns.scatterplot(
    data=df,
    x="petal length (cm)",
    y="petal width (cm)",
    hue="species"
```

```
)

plt.title("Relación entre longitud y anchura del pétalo")
plt.show()

# 3. FIGURA 2 - DIAGRAMA DE CAJA

sns.boxplot(
    data=df,
    x="species",
    y="petal length (cm)"
)

plt.title("Distribución de la longitud del pétalo por especie")
plt.show()

# 4. MODELO SUPERVISADO - SVC

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

modelo = make_pipeline(
    StandardScaler(),
    SVC(kernel="linear", C=1)
)

# Entrenamiento
modelo.fit(X_train, y_train)

# Predicciones
pred = modelo.predict(X_test)

# Resultados
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))

# 5. FIGURA 3 - MATRIZ DE CONFUSIÓN

ConfusionMatrixDisplay.from_predictions(
    y_test,
```

```
    pred,
    display_labels=iris.target_names
)

plt.title("Matriz de confusión - SVC")
plt.show()

# 6. VALIDACIÓN CRUZADA DEL MODELO SUPERVISADO

scores = cross_val_score(
    modelo,
    X,
    y,
    cv=5
)

print("Validación cruzada:", scores)
print("Media:", scores.mean())

# 7. MODELO NO SUPERVISADO - K-MEANS

escalador = StandardScaler()

X_escalado = escalador.fit_transform(X)

kmeans = KMeans(
    n_clusters=3,
    random_state=0,
    n_init=10
)

clusters = kmeans.fit_predict(X_escalado)

# Resultados de K-Means
print("Etiquetas:", kmeans.labels_)
print("Centroides:", kmeans.cluster_centers_)
print("Inercia:", kmeans.inertia_)
print("Iteraciones:", kmeans.n_iter_)

# Coeficiente Silhouette
print(
    "Silhouette:",
    silhouette_score(X_escalado, clusters)
)
```

```
# 8. FIGURA 4 - MÉTODO DEL CODO

inercias = []

for k in range(1, 9):

    modelo_k = KMeans(
        n_clusters=k,
        random_state=0,
        n_init=10
    )

    modelo_k.fit(X_escalado)

    inercias.append(modelo_k.inertia_)
plt.plot(
    range(1, 9),
    inercias,
    marker="o"
)

plt.xlabel("Número de agrupamientos K")
plt.ylabel("Inercia")
plt.title("Método del codo para K-Means")

plt.show()

# 9. PCA PARA REDUCIR LOS DATOS A DOS DIMENSIONES
pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_escalado)

# 10. FIGURA 5 - REPRESENTACIÓN DE LOS CLUSTERS K-MEANS

plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=clusters
)
plt.xlabel("Componente principal 1")
plt.ylabel("Componente principal 2")
plt.title("K-Means sobre Iris")

plt.show()
```

Referencias utilizadas

• Material didáctico de SEAS, Machine Learning en Python, Unidades 1 a 8.

• Enunciado y criterios de evaluación del Trabajo Obligatorio de Machine Learning de SEAS.

Nota: los resultados numéricos y las figuras de este documento se han generado ejecutando el código sobre el conjunto Iris incluido en Scikit-learn.